Entendendo o artigo CheckList para apresentar

> **Conteúdo gerado por IA (Claude).** Escrito a partir da leitura do PDF completo do artigo. Revise antes de apresentar.

Artigo: Ribeiro, M. T.; Wu, T.; Guestrin, C.; Singh, S. *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*. ACL 2020, p. 4902–4912. **PDF:** https://aclanthology.org/2020.acl-main.442.pdf (11 páginas) **Slides do seminário:** https://claude.ai/artifact/H2kUtV9vqwkdLNNRsoDxKc

---

A ideia em 30 segundos

Modelos de linguagem costumam ser avaliados por **um número**: a acurácia num conjunto de teste. Um modelo com 92% parece ótimo. O artigo mostra que esse número esconde falhas graves. O mesmo modelo pode errar quase sempre frases simples como *"I thought the plane would be awful, but it wasn't"* ("achei que o avião seria horrível, mas não foi").

A proposta, chamada **CheckList**, é testar modelos de linguagem **como se testa software**: uma habilidade de cada vez, com muitos casos de teste gerados automaticamente.

A frase para guardar: **o CheckList troca a pergunta "quanto o modelo acerta?" por "o que o modelo realmente sabe fazer?"**

Parte 1 — Contribuições e trabalhos relacionados

O problema que motivou o artigo

A forma padrão de avaliar um modelo de PLN (Processamento de Linguagem Natural) é:

1. separar os dados em treino, validação e teste;
2. treinar o modelo;
3. medir a acurácia no conjunto de teste, muitas vezes num ranking público (*leaderboard*).

Os autores apontam três defeitos nisso:

- **O conjunto de teste tem os mesmos vieses do treino.** Por isso o desempenho no mundo real pode ser bem pior que o medido.
- **Um número só não diz onde o modelo erra** nem como corrigir.
- **As alternativas que existiam eram fragmentadas:** cada uma olhava uma tarefa ou uma habilidade (só robustez, só justiça…). Faltava um guia geral do que testar.

As contribuições (slide 2)

#	Contribuição	Em palavras simples
1	**Metodologia**	Traz o *teste comportamental* (caixa-preta) da engenharia de software para modelos de linguagem. Serve para qualquer tarefa e qualquer modelo.
2	**Matriz de capacidades**	Uma lista de 10 habilidades linguísticas que diz **o que** testar: negação, nomes próprios, correferência…
3	**Três tipos de teste**	MFT, INV e DIR dizem **como** testar. Dois deles (INV e DIR) não precisam de dados rotulados.
4	**Ferramenta de código aberto**	Gera milhares de casos de teste com modelos de frase, sugestões do RoBERTa e perturbações prontas. Disponível em github.com/marcotcr/checklist.
5	**Bugs encontrados**	Em 3 tarefas, falhas de até 100% em modelos de pesquisa (BERT, RoBERTa) e em serviços comerciais (Microsoft, Google, Amazon).
6	**Evidência com usuários**	Quem usou o CheckList criou 2× mais testes e achou ~3× mais bugs. A própria equipe da Microsoft achou bugs novos num modelo já muito testado.

Como resumir na fala: “O artigo entrega um método, uma ferramenta e a evidência de que esse tipo de teste acha bugs que a acurácia esconde.”

Trabalhos relacionados (slide 3)

A mensagem central: **cada abordagem anterior resolve um pedaço do problema; o CheckList junta tudo num roteiro só.**

Abordagem anterior	O que faz	Limitação	O que o CheckList faz diferente
Acurácia em benchmark (SST-2, QQP)	Mede acertos num conjunto de teste	Dados enviesados; um número só	Dá uma nota por habilidade
Testes de robustez (erros de digitação, ruído, exemplos adversários)	Testa se o modelo aguenta mudanças	Olha uma habilidade só	Robustez vira apenas uma linha da matriz
Testes de justiça e consistência (troca de nomes, checagens lógicas)	Testa vieses e coerência	Feitos para uma tarefa de cada vez	Os mesmos testes servem para várias tarefas
Conjuntos de desafio (*challenge sets*)	Exemplos feitos à mão para um fenômeno	Pequenos, artificiais, focados em casos difíceis	Modelos de frase dão controle; dados reais dão escala; foca em casos que **deveriam ser fáceis**
Sondas (*probes*)	Treinam um classificador sobre o "interior" do modelo	Mostram o que o modelo codifica, não como se comporta na tarefa	Testa o comportamento final
Teste de software (Beizer 1995; Segura et al. 2016)	Teste caixa-preta e teste metamórfico	— (é a inspiração)	MFT = teste unitário; INV e DIR = testes metamórficos

Exemplo bom para falar sobre sondas: uma sonda sobre o BERT reconhece nomes próprios com 96,7% de acerto. Mas o BERT ajustado para a tarefa do Quora falha em testes básicos com nomes. Saber algo "por dentro" não garante usar isso na tarefa.

O que o CheckList não cobre, segundo os próprios autores: erros de rótulo, versão dos dados, ataques de segurança e falta de interpretabilidade.

Parte 2 — Conceitos que você precisa entender

Acurácia × teste comportamental (slide 4)

- **Acurácia em dados reservados (*held-out accuracy*):** porcentagem de acertos em exemplos que o modelo não viu no treino. Exemplo: o BERT tem 92,7% no SST-2 (resenhas de filmes).
- **Teste comportamental (caixa-preta):** testar só pela entrada e saída, sem olhar código ou pesos. Cada teste verifica um comportamento.

Analogia: a acurácia é a **média final** do aluno. O teste comportamental é o **boletim por matéria**. Um aluno com média 9 pode estar tirando zero em uma matéria.

Capacidade, perturbação e teste metamórfico (slide 5)

Conceito	O que é	Exemplo
Capacidade	Uma habilidade que a tarefa exige	Negação: “not poor” (não é ruim) não é negativo
Perturbação	Uma mudança controlada na entrada	Trocar “Chicago” por “Dallas” num tweet
Teste metamórfico	Compara saídas entre si, sem precisar do gabarito	Se só troquei a cidade, o sentimento tem que continuar igual

Por que o teste metamórfico é importante: eu não sei qual é o sentimento “certo” de um tweet qualquer, mas sei que trocar o nome da cidade não deveria mudá-lo. Então dá para testar com dados reais **sem rótulo**.

Analogia: é como conferir uma balança. Sem saber seu peso exato, você sabe que tirar o relógio não pode fazer o número subir.

As tarefas e os modelos (slide 6)

Tarefa	O que o modelo precisa fazer
Análise de sentimento	Dizer se um texto é positivo, negativo ou neutro
Pares de perguntas do Quora (QQP)	Dizer se duas perguntas perguntam a mesma coisa
Compreensão de leitura (SQuAD)	Responder a uma pergunta a partir de um parágrafo

- **BERT e RoBERTa:** modelos de linguagem do tipo Transformer, pré-treinados com muito texto e depois ajustados (*fine-tuned*) para cada tarefa.
- **Modelo de linguagem mascarado:** adivinha a palavra escondida numa frase. Com “I really [mask] the flight”, sugere *enjoyed*, *liked*, *hated*... O CheckList usa isso para ajudar a gerar testes.

Parte 3 — Metodologia

A matriz (slide 7)

O teste é organizado numa tabela:

- **linhas** = capacidades (vocabulário, nomes próprios, negação...);
- **colunas** = tipos de teste (MFT, INV, DIR);
- **cada célula** = a taxa de falha, ou seja, a porcentagem de casos que o modelo errou.

Exemplo do artigo (Figura 1), com um modelo comercial de sentimento:

Capacidade	MFT	INV	DIR
Vocabulário	15,0%	16,2%	34,6%
Nomes próprios	0,0%	20,8%	—
Negação	**76,4%**	—	—

Ou seja: o modelo erra 76,4% de um teste simples de negação.

Os três tipos de teste (slide 8)

Tipo	Ideia	Exemplo	O que deve acontecer
MFT – teste de funcionalidade mínima	Casos simples com resposta certa (como um teste unitário)	"I didn't love the flight."	Negativo
INV – teste de invariância	Uma mudança que **não** deveria importar	Trocar "Cuba" por "Canada"	Mesma previsão
DIR – teste de expectativa direcional	Uma mudança com efeito conhecido	Acrescentar "You are lame."	O sentimento não pode ficar mais positivo

Ponto-chave: INV e DIR **não precisam de rótulo**. Comparam a resposta antes e depois da mudança.

Exemplo do dia a dia (um app que lê avaliações de restaurante):

- **MFT:** "A comida não estava ruim" tem que sair positivo ou neutro, nunca negativo.
- **INV:** "restaurante em Curitiba" → "restaurante em Recife" não pode mudar a nota.
- **DIR:** acrescentar "Péssimo atendimento" não pode deixar a avaliação melhor.

Gerando muitos testes (slide 9)

Escrever milhares de testes à mão é inviável. A ferramenta usa **modelos de frase** (*templates*):

```
I {NEGATION} {POS_VERB} the {THING}.
{NEGATION} = didn't, can't say I, ...
{POS_VERB} = love, like, ...
{THING}    = food, flight, service, ...
```

Cada combinação vira um caso de teste: com listas pequenas, já são centenas de frases.

A ferramenta também:

- usa o **RoBERTa para sugerir palavras** para as listas;
- traz **listas prontas** de nomes de pessoas, cidades, países e grupos;
- traz **perturbações prontas**, como trocar letras vizinhas para simular erro de digitação;
- permite **salvar e compartilhar** as suítes de teste.

Analogia: o modelo de frase é uma fábrica. “Eu {não gostei / não recomendo} do {comida / atendimento / preço}” gera 2 × 3 = 6 frases de uma vez.

Desenho experimental (slide 10)

Parte A — testar os modelos

Tarefa	Modelos testados	Desempenho no benchmark
Sentimento	APIs pagas da Microsoft, do Google e da Amazon	não informado
Sentimento	BERT e RoBERTa ajustados no SST-2	92,7% e 94,8%
Pares do Quora	BERT e RoBERTa ajustados no QQP	91,1% e 91,3%
Compreensão de leitura	BERT-large ajustado no SQuAD	93,2 de F1

- Para sentimento, os testes INV e DIR usaram **tweets sobre companhias aéreas, sem rótulo**.
- **Quando um teste falha:**
  - no MFT, quando a resposta é diferente da esperada;
  - no INV e no DIR, quando a previsão muda (ou vai na direção proibida) **mais de 0,1** em probabilidade. Isso evita contar oscilações mínimas.

Parte B — testar com pessoas

1. **Equipe da Microsoft:** sessão de cerca de 5 horas com a equipe que mantém o modelo comercial de sentimento. Os autores apresentaram o método **sem mostrar os testes deles**. A equipe pensou em uns 30 testes e implementou uns 20.
2. **Experimento controlado:** 18 pessoas com experiência intermediária em PLN (8 da indústria, 10 da academia), 2 horas para testar o BERT no Quora, divididas em três grupos de 6:
   - **sem ajuda;**
   - **só com a lista de capacidades;**
   - **com a ferramenta completa.**

   Cada pessoa deu uma nota de **gravidade** aos bugs, de 1 (“não é bug”) a 5 (“nenhum modelo com esse bug deveria estar em produção”).

Parte 4 — Resultados e aplicações

Os modelos falham no básico (slide 11)

Número	O que aconteceu
100%	Negação no fim da frase (“…would be awful, but it wasn’t”): Microsoft e Amazon erram todos os casos
100%	Compreensão de leitura: erra todos os testes simples de “ele”/“ela” (“He is a journalist, and she is an adviser. Who is a journalist?”)
98%	Pares do Quora: o BERT confunde “before” (antes) com “after” (depois)
89%	“John is not a doctor, Mary is. Who is a doctor?” → o modelo responde John. Com os nomes trocados, erra só 3,2%: **viés de gênero**

Outras descobertas importantes:

- **Os modelos de pesquisa foram melhores que os comerciais** na maioria dos testes, o que surpreendeu os autores. A exceção são as frases neutras: o BERT e o RoBERTa erram quase todas porque foram treinados só com positivo e negativo.
- **A acurácia escondia o problema:** o BERT acerta mais de 90% das frases com negação do próprio benchmark SST-2, mas falha nos testes simples de negação do CheckList.
- **Interpretação dos autores:** os modelos usam **atalhos**, por exemplo se apoiar nos nomes próprios, em vez de entender de verdade.

Quem usou o CheckList achou mais bugs (slide 12)

Comparando o grupo com a ferramenta completa e o grupo sem ajuda:

Medida	Sem ajuda	Com CheckList	Diferença
Testes criados	5,8	13,5	**2×**
Casos por teste	7,3	198	**27×**
Bugs graves (gravidade ≥ 3)	2,2	6,2	**≈3×**

- Uma pessoa de fora reavaliou a gravidade dos bugs e chegou a resultados quase iguais.
- A **equipe da Microsoft** achou bugs novos num modelo maduro. Também testou coisas que não tinha pensado, como hashtags (“#IHateYou”) e negação implícita (“I wish it was good”), e disse que vai adotar o CheckList.
- **Ressalva:** cada grupo tinha só 6 pessoas e o artigo não traz teste estatístico. É uma indicação forte, não uma prova.

Aplicação ao nosso projeto (slide 13)

O nosso projeto é **fuzzing com gramática de modelos de chat do Hugging Face**. As ideias do CheckList se encaixam quase uma a uma:

CheckList	No nosso projeto
Modelos de frase (*templates*)	Nossa gramática gera os prompts
MFT	Tarefas com resposta conhecida: contas, contar letras, inverter texto, JSON
INV	Paráfrases da mesma pergunta têm que receber a mesma resposta
Capacidades	Uma categoria por habilidade, por exemplo copiar texto Unicode exatamente
DIR	Ainda não usamos: trabalho futuro

Exemplo real dos nossos experimentos: perguntamos ao Qwen 2.5 quantas letras “r” tem em “strawberry”, de três formas diferentes. Ele respondeu **3, 1 e 10**. É exatamente uma falha de invariância, o tipo de problema que o CheckList foi criado para revelar.

Conclusão (slide 14)

- **A acurácia é útil, mas não basta.**
- **Pontos fortes:** o CheckList serve para qualquer tarefa e modelo, acha bugs até em produtos comerciais e é fácil de aprender.
- **Limitações:**
  - testa só comportamento (não pega erro de rótulo nem problema de segurança);
  - **passar num teste não prova que o modelo está certo** (os comerciais passaram no teste simples de justiça, e isso não prova que são justos);
  - o estudo com usuários é pequeno (essa é uma observação nossa, não dos autores).
- **Trabalho futuro proposto pelos autores:** criar suítes de teste em colaboração, já que muitos testes servem para várias tarefas.

Analogias para usar na apresentação

- **Boletim escolar:** a acurácia é a média; o CheckList é a nota por matéria.
- **Inspeção de carro:** ninguém aprova um carro só porque ele andou 100 km; existe uma lista (freio, farol, pneu). A matriz é essa lista para modelos de linguagem.
- **Aluno que decorou palavras-chave:** acerta a prova de sempre, erra quando um detalhe muda (“before” por “after”). É o que os autores chamam de atalho.
- **Balança:** sem saber seu peso exato, você sabe que tirar o relógio não pode aumentar o número. É assim que INV e DIR funcionam sem gabarito.

Perguntas que podem aparecer (com respostas curtas)

1. **Qual o problema da acurácia?** Superestima o desempenho real, tem os vieses do treino e não diz onde o modelo erra.
2. **O que é teste comportamental?** Testar só pela entrada e saída, sem olhar o interior do modelo. Por isso dá para testar APIs comerciais.
3. **Qual a diferença entre MFT, INV e DIR?** MFT: caso simples com resposta certa. INV: mudança que não deve alterar a resposta. DIR: mudança com efeito conhecido na resposta.
4. **Como testar sem rótulo?** INV e DIR comparam a resposta com a resposta anterior do próprio modelo, não com um gabarito.
5. **Quais foram as falhas mais graves?** Negação (perto de 100%), correferência (100%), noção de tempo (“before”/“after”) e papéis semânticos (quem fez o quê).
6. **Por que os modelos de pesquisa foram melhores que os comerciais?** O artigo trata isso como surpresa e não dá uma explicação fechada.
7. **Os modelos comerciais são justos, já que passaram no teste de justiça?** Não dá para concluir isso. Passar num teste simples não prova ausência de viés.
8. **A gravidade dos bugs não é subjetiva?** É. Para reduzir isso, uma pessoa de fora reavaliou e o resultado foi quase igual. Mas com 6 pessoas por grupo é uma indicação, não prova.
9. **Em que o CheckList difere das sondas?** Sondas olham o interior do modelo; o CheckList testa o comportamento na tarefa final.
10. **Quais as limitações?** Não cobre erros de rótulo, versão dos dados, ataques nem interpretabilidade. E depende da criatividade de quem escreve os testes.
11. **Como isso se liga ao projeto de vocês?** Nossa gramática faz o papel dos templates, nossas tarefas com resposta conhecida são MFT e nosso teste de paráfrase é INV.

Antes de apresentar, confira no PDF

- O artigo ganhou o prêmio de melhor artigo da ACL 2020. Essa informação não está no texto do PDF; confirme antes de citar.
- Qual modelo previu sempre negativo no teste de justiça para grupos como *black* e *gay*. O símbolo do modelo não saiu na extração do texto; pela ordem do parágrafo, é um dos modelos de pesquisa.